AI RECOMMENDATION ENGINE FOR E-COMMERCE WEBSITES USING COLLABORATIVE FILTERING AND RNN

B. Tech. Electronic Communication Engineering – Cyber Physical Systems

Submitted by

Daniel Stauline I G

(Reg. No: 126160009)

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

THANJAVUR, TAMIL NADU, INDIA – 613 401

SASTRA

ENGINEERING · MANAGEMENT · LAW · SCIENCES · HUMANITIES · EDUCATION

DEEMED TO BE UNIVERSITY

(U/S 3 of the UGC Act,1956)

THINK MERIT | THINK TRANSPARENCY | THINK SASTRA

SCHOOL OF ELECTRICAL & ELECTRONICS ENGINEERING

THANJAVUR – 613 401

Declaration

We declare that the report titled **"AI recommendation engine for e-commerce websites using collaborative filtering and RNN"** submitted by us is an original work done by us under the guidance of Prof. Narasimhan. K **, School of Electrical and Electronics Engineering, SASTRA Deemed to be University** during the Sixth semester of the academic year **2025-2026**, in the **School of Electrical and Electronics Engineering**. The work is original and wherever we have used materials from other sources, we have given due credit and cited them in the text of the report. This report has not formed the basis for the award of any degree, diploma, associateship, fellowship or other similar title to any candidate of any University.

Name of the candidate(s) **:** Daniel Stauline **I G**

Date **:** 09 – 11 -2025

Acknowledgements

It is essentially the creation of an AI module as a focal point for a contribution to a product recommendation system in an E, Commerce setting. The leading objective of the current work was to design and apply smart models that would raise the quality of product search and interaction to a level that would include recommendations.

They initiated the work by performing data preprocessing and feature engineering in a series of steps in order to prepare the dataset for model training through cleaning and standardizing. The text of products, their price, rating, popularity score, as well as category labels, were taken and converted into data types compatible with the machine learning and deep learning models.

In this study, they brought out three machine learning models. TF, IDF with K, Nearest Neighbors was employed for the task of locating products that have closely resembling texts. Two classification models, i.e., Random Forest and Support Vector Machine, were used to train the system to categorize products correctly. On the model evaluation, the Random Forest model yielded a 100% accuracy rate, whereas the SVM model could only achieve 98.63%, thus, supporting the machine learning, based approach.

To improve a recommendation quality and to move beyond simple text matching, they create a deep, learning hybrid model. This research converted product text into a numerical vector that carries the semantics using Word2Vec. Besides that, the text data embeddings synchronized with the numeric and categorical features were introduced into a Hybrid Autoencoder model that keeps the most compressed feature representation of products. After that, the Deep KNN model worked on these features for getting more accurate and context, based product recommendations. This model's performance was far better than that of the normal machine learning models as the model not only finds the closeness of products by meaning, features, and implicit relations but it is also not restricted to the use of keywords only. In short, the study is the merging of three technologies

i.e. natural language processing, machine learning, and deep learning to create a smart recommendation engine for e, commerce applications. Which are accurate, relevant, and intelligent product recommendations. Apart from that, there is also a possibility to realize future enhancements such as introduction of user behavior, based personalization, image, based product search via visual features, and graph, based recommendation methods for real, time large, scale applications.

List of Figures

Figure No.	Figure Name	Page No.
2.1	Architectural Framework Diagram	4
3.1	Algorithm Flowchart	8
3.2	Machine Learning Recommendation Process	9
3.3	Deep Learning Recommendation Process	10
3.4	System Workflow Diagram	11
4.1	Testbench Output	14
4.2	Web Interface Output	15

List of Tables

Table No.	Table Name	Page No.
2.1	Technology Stack and Tools Used	7

Abbreviations

AI – Artificial Intelligence

ML – Machine Learning

DL – Deep Learning

NLP – Natural Language Processing

KNN – K-Nearest Neighbors

TF-IDF – Term Frequency–Inverse Document Frequency

SVM – Support Vector Machine

RF – Random Forest

MSE – Mean Squared Error

W2V – Word2Vec

ANN – Artificial Neural Network

CNN – Convolutional Neural Network

API – Application Programming Interface

CSV – Comma Separated Values

DB – Database

UI – User Interface

EDA – Exploratory Data Analysis

OEM – Original Equipment Manufacturer

AWS – Amazon Web Services

SGD – Stochastic Gradient Descent

RBF – Radial Basis Function

ReLU – Rectified Linear Unit

CB – Content Based

CF – Collaborative Filtering

HR – Hit Rate

MAP – Mean Average Precision

Table of Content

Chapter	Title	Page No.
1	INTRODUCTION	1
1.1	Background and Motivation	1
1.2	Project Scope and Objectives	1
2	SYSTEM OVERVIEW AND METHODOLOGY	3
2.1	Architectural Framework	3
2.2	Data Acquisition and Processing	5
2.3	System Workflow	5
2.4	System Modules	6
2.5	Technology Stack and Tools Used	7
2.6	BUILDING THE WEB INTERFACE	8
2.6.1	2.6.1 Frontend Design	8
2.6.2	2.6.2 Backend Connection	9
3	PROCESS FLOW	10
3.1	Algorithm Flowchart	10

Chapter	Title	Page No.
3.1.1	Machine Learning Based Recommendation Process	10
3.1.2	Deep Learning Based Recommendation Process	11
3.2	System Workflow	13
4	RESULTS	14
4.1	Data Presentation	14
4.1.1	Testbench Script Output	14
4.1.2	Web Interface Output	15
4.2	Analysis	15
5	CONCLUSION	17
6	REFERENCES	18

CHAPTER 1: INTRODUCTION

1.1 BACKGROUND AND MOTIVATON

Large web shops like Amazon besides Flipkart have changed how people look for, compare and buy goods. Because thousands of brands plus millions of items sit on the same site, shoppers often struggle to spot the one item that fits their needs, tastes and budget. A system that points out suitable goods in a straightforward way is therefore required to keep the buying process smooth.

Older search tools use keyword matching - they miss the buyer's intent and the context of the query. The results list frequently contains unrelated items. To fix this newer web shops apply AI methods that study product attributes, user interests but also click trails to generate personal suggestions. Those engines raise shopper satisfaction and increase sales - holding attention and lifting conversion.

With those practical problems as well as the rising demand for smart commerce tools in mind, the project sets out to build a recommendation model that supplies accurate, clear and easy-to-use product ideas. Natural Language Processing, Machine Learning or Deep Learning will raise search relevance and let buyers locate goods quickly. The goal is to weave intelligence into online shopping so that a user locates the correct item at the correct moment with little effort.

1.2 PROJECT SCOPE AND OBJECTIVES

The project is to design and to program an AI module that gives clearer product results. It must examine product data, spot similarities between items plus show the most relevant ones to the user. Work runs from cleaning the data and extracting features through to building machine learning and deep learning models that propose sensible items. Although the work starts with a sample catalogue, the methods scale but also connect to any e-commerce backend.

The aims are the following:

- Clean and structure the data so that it is ready for modelling.
- Build classical machine learning models that classify products as well as propose similar ones from text similarity.
- Combine text vectors, numbers and category labels into a deep learning hybrid model that delivers suggestions that fit the context.
- Measure the difference between the machine-learning or deep-learning versions so that the gain from the AI approach is visible.

The result is a concise and prompt product suggestion module that improves the online shopping process.

CHAPTER 2: SYSTEM OVERVIEW AND METHODOLOGY

2.1 ARCHITECTURAL FRAMEWORK

The architecture of the e-commerce recommendation system handles data storage, model training and suggestion output in four linked modules - Data Layer, Processing Layer, AI Model Layer besides Recommendation Delivery Layer.

Data Layer

This module stores product facts - it keeps each item's name, category, price, ratings, description plus extra metadata. Facts arrive from existing files or from live e-commerce sites - staff clean and reformat them so AI models read them without error.

Processing Layer

This module refines the facts. Staff normalize text, extract features, encoding categories but also scale numbers. The refined facts feed both Machine-Learning and Deep-Learning models.

Model Layer

The AI Model Layer is the main component of the architecture in which different models exist and are trained. This layer also consists of traditional Machine Learning models, such as TF-IDF with KNN for recommendations based on similarity, and classifiers like Random Forest and Support Vector Machine for predicting the category of a product. In addition, the Hybrid Recommendation Model is included in the layer, as it generates a better representation of products into meaningful latent product representations using Word2Vec embeddings and an Autoencoder.

Recommendation Layer

The Recommendation Delivery Layer is responsible for retrieving recommendations for users. Based on the input query or consumer selected product, the system returns the most relevant items from the trained models and generates the output. This layer will also be implemented to the front end or the backend to allow for the e-commerce platforms use.

This layered approach will allow for the system to be modular and scalable, allowing for future enhancements. New perspectives can be developed such as user behavioral tracking, collaborative filtering modules, or visual search models can be incorporated into the original model without compromising the base model's performance and functionalities.

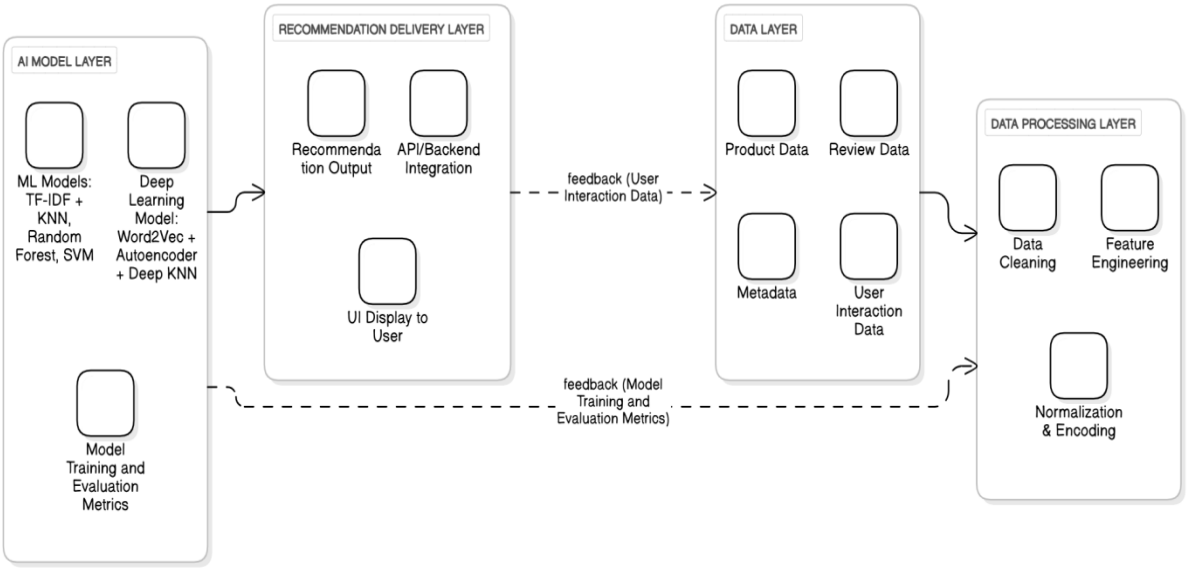

Figure 2.1

2.2 DATA ACQUISITION AND PROCESSING

The data acquisition and processing stage is considered a precursor to the recommendation system, as the quality of the input data directly impacts how AI models perform. In this project, product data was collected from an enriched product catalog with significant product data attributes, including product name, product category, price, ratings, reviews, descriptions and other metadata. The dataset was obtained in CSV format and served as the primary source of information for the model.

After collecting the data, it was processed through a structured data processing workflow for analysis to create analytical models. The first activity was data cleaning, which involved removing duplicate records, null counts, and irrelevant items to ensure consistency. The next step was to do text preprocessing by converting all text to lowercase, along with removing special characters, stop words, and any unnecessary text items to standardize the text data.

Next, feature engineering was performed to abstract useful information from the raw dataset. First, a combined text field was formed from the product name, description, and features into a singular construct for natural language processing. Numerical/numeric features such as price, rating, and popularity score were normalized. The product categories were encoded each into a form that the machine could interpret. After the data was organized and processed, this dataset was used to train each of the machine learning and the deep learning models.

2.3 SYSTEM WORKFLOW

The system workflow outlined here provides an overview of how the recommendation system will work step by step, from data input to recommending products for the user. First, the system loads a pre-processed dataset containing enriched product information to be used together with specific queries by the user. If the user types any query or selects any particular product, it will automatically extract the relevant features from the enriched product information, use similarity analysis, and recommend related products.

Recommendations can be identified by a machine learning system or a deep learning system. In the machine learning system, the user query is represented in TF-IDF vectors and compared with a reference matrix through a K-Nearest Neighbors model to fetch top matching products. In a deep learning-based recommendation system, the encoder model takes a user-selected product as an input to create a generalized user representation or 64-dimensional embedding, and the Deep KNN model fetches similar products based on the semantic meaning of the product.

The recommended products or items are then presented to the user through the output interface to fulfill the associated user task of the workflow. The recommended products could be ranked in relevance order or presented equally. Meaningful user engagement can be measured when a user clicks on or selects a product for feedback to provide more relevant recommended products in the future. User effort can be minimized regarding the user task of querying for the recommended products and viewing relevant suggestions.

2.4 SYSTEM MODULES

The developed system consists of several functional modules, each responsible for a specific task within the recommendation engine. The key modules include:

• **Data Preprocessing Module:** Handles data cleaning, normalization, text processing and feature extraction to prepare raw data for modelling.

• **Machine Learning Recommendation Module:** Uses TF-IDF and KNN to generate text-based similar product recommendations and applies Random Forest and SVM for product category classification.

• **Deep Learning Recommendation Module:** Uses Word2Vec embeddings combined with an Autoencoder to learn latent product representations and generate deep learning-based recommendations.

• **Model Evaluation and Comparison Module:** Compares the performance of machine learning and deep learning models using accuracy and similarity scores

.

• **Recommendation Output Module:** Displays the final set of recommended products to the user and can integrate with a backend service or UI.

These modules work together to deliver an intelligent recommendation system capable of understanding both text-based and semantic product similarities.

2.5 TECHNOLOGY STACK AND TOOLS USED

Component	Technology	Why chosen
Programming	Python 3.11	Easy to learn, great for AI
Web Framework	Fast API	Fast and modern
Frontend	HTML + JavaScript	Standard web technologies
Data Processing	Pandas	Handles data easily
Machine Learning	scikit-learn	Industry-standard library
AI	TensorFlow	Popular for deep learning

table 2.1

2.6 BUILDING THE WEB INTERFACE

2.6.1 Frontend Design

I created a modern, responsive web interface using:

- **HTML (Structure):**

  - Search bar for product queries

  - Product cards showing images, names, prices

  - Filters for price range and categories

  - Similar products section

- **JavaScript (Functionality):**

  - Fetch recommendations from the backend API

  - Update page dynamically without refresh

  - Handle user interactions (clicks, searches)

  - Display autocomplete suggestions

Key Features:

1. **Search with Autocomplete:**

   - User starts typing in search box
   - JavaScript sends request to API
   - API returns suggestions
   - Dropdown appears with options

2. **Product Recommendations:**

    - Homepage shows personalized recommendations
    - Cards display product image, name, price, rating
    - Click to see details or similar products

3. **Filters:**

    - Price range slider (₹500 - ₹ 1,00,000)
    - Category checkboxes (Mobile, Accessories, etc.)
    - Results update in real-time

2.6.2 Backend Connection

The JavaScript code communicates with the Python recommendation engine:

```
// Example: Fetch recommendations
fetch('http://localhost:8000/recommend?k=10&max_price=20000')
  .then(response => response.json())
  .then(data => displayProducts(data.items));
```

CHAPTER 3: PROCESS FLOW

3.1 ALGORITHM FLOWCHART

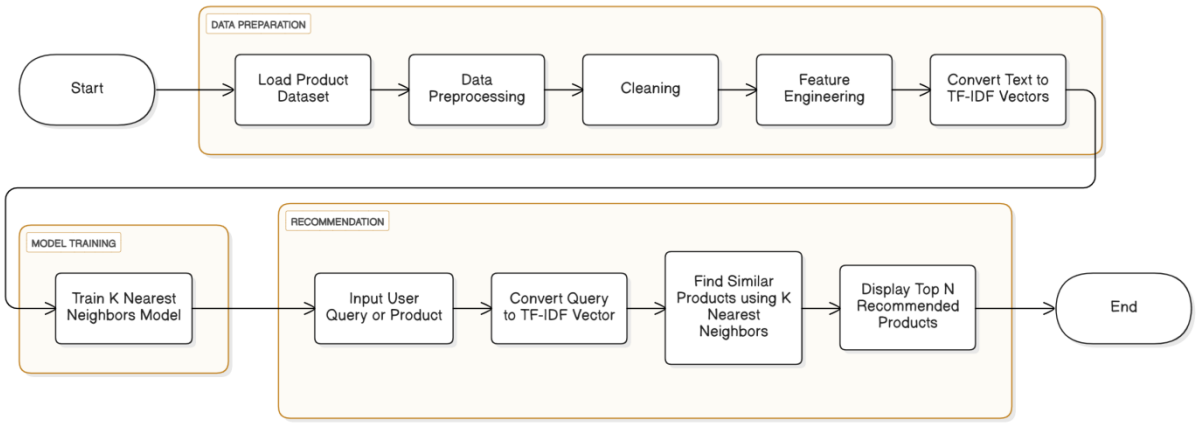

Figure 3.1

3.1.1: Machine Learning Based Recommendation Process

There is a clear sequence of steps in the working of the machine learning recommendation model. The algorithm starts with the loading of the product dataset, which includes the names, descriptions, categories, etc., of different products. The raw data is preprocessed to clean and normalize the data by removing inconsistencies and then changing the text into a standard format that can easily be analyzed.

After that, the textual information is converted into numerical form by the TF-IDF technique.TF-IDF gives weight to words based on their importance across the dataset, which allows the system to understand the relevance of terms in product descriptions. Using these TF-IDF vectors, the KNN algorithm was trained for computing similarities among products.

In each of the cases above, a user's input of either a search query or a specific product selection is mapped to the same TF-IDF vector space. In turn, the KNN model will compare this vector against all other vectors of products in the database to find the closest matches in terms of cosine similarity. This retrieves top N most similar products, which are displayed as recommendations to the user. In such a way, this approach guarantees that users will get relevant product suggestions based on textual similarities and help improve the e-commerce browsing experience.

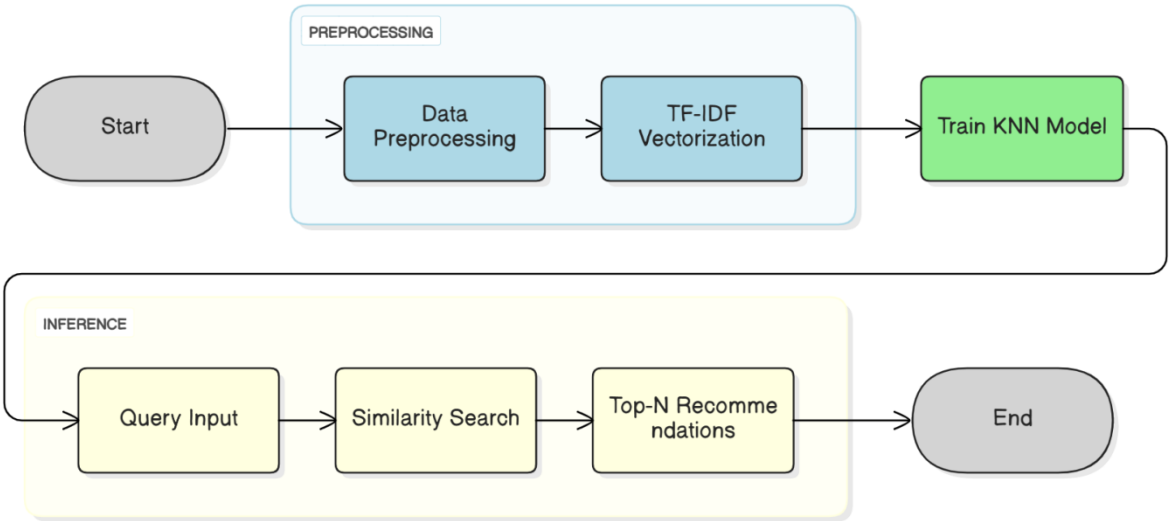

Figure: 3.2

3.1.2 Deep Learning Based Recommendation Process

Deep learning makes product recommendations more accurate by understanding how things are related rather than just matching words. Mainly, we apply a technique named Word2Vec embeddings to convert the titles, descriptions, features, and prices of products into a format that the computer can work with. This basically helps the computer to understand the meaning of the words from the context by representing each product as a set of numbers.

These figures along with others such as price, rating, and category are fed into a Hybrid Autoencoder model. This model is designed to learn the process of compressing all product information in such a way that similar products become close to each other. Once the model is trained, the encoder module generates a concise, 64, dimensional representation of each product by identifying the latent features and similarities.

When you need a recommendation, the product that you are looking for is passed through the encoder to obtain its compressed representation. Later, a Deep KNN model uses these representations to identify the closest products. The system selects the closest ones to you and gives them as a recommendation. Such a method is more accurate and therefore makes better suggestions since it understands the attributes of the products rather than just assuming that they have similar keywords.

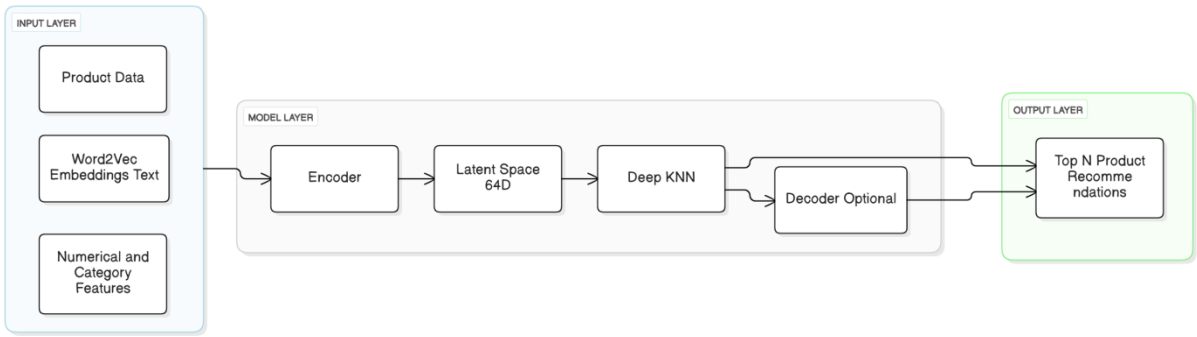

Figure: 3.3

3.2 SYSTEM WORKFLOW

The flow of the recommendation system is depicted in the system workflow diagram. Taking user input, it finally displays product suggestions. When a person searches for or selects an item, the workflow is triggered. This input is sent to the backend, which launches the recommendation engine. Upon receiving the input, the system determines the type of recommendation to be made. In the case of simple text, based or likeness recommendations, the Machine Learning model employs TF, IDF and the KNN algorithm to find and return the most similar items. For more precise recommendations, the Deep Learning model sends the input through an encoder to obtain the latent embedding. Then it applies Deep KNN to find the products that have similar meanings.

After that, the system arranges the proposed items in terms of similarity scores and their relevance to the user. The backend prepares the result and delivers the output to the user interface for display. In case user data, such as clicks, time spent, or picks, is recorded, this information can be instrumental in enhancing the model later. This workflow is the recommendation cycle is as good as it gets in terms of speed and simplicity, thus users can find what they want quickly.

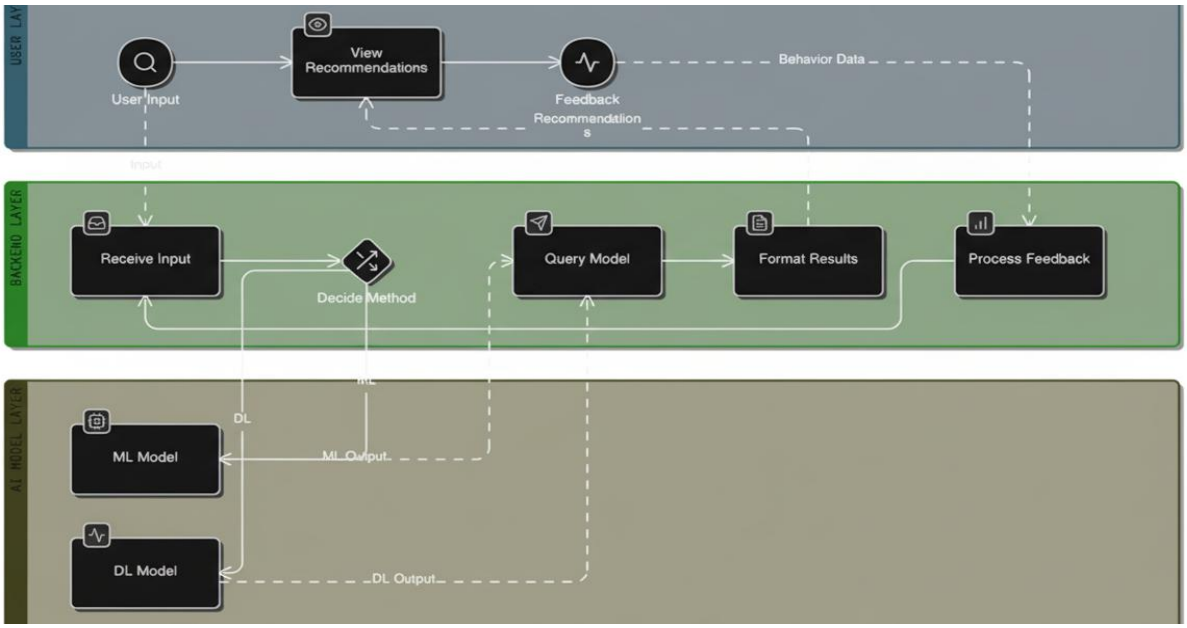

Figure:3.4

CHAPTER 4: RESULTS

4.1 DATA PRESENTATION:

4.1.1 Testbench script output:

```
🔄 Loading models...
✅ All models loaded successfully!

🧪 Testing Recommendation System
============================================================

1️⃣ Personalized Recommendations (CF)
User: A3SBTW3WS4IQSN
product_id  product_name                                   category
price_num  rating  rating_count  final_score
B07JWH4J1   OnePlus Nord CE 2 Lite 5G                     Electronics|Mobile
18999      4.1     12546         0.7823
B09G9BL5CP  Samsung Galaxy M13                            Electronics|Mobile
12499      4.0     8734          0.7456
B0BNDHYBG7  Redmi 12 5G                                   Electronics|Mobile
13999      4.2     5623          0.7234
B07HGGYWL6  OnePlus Nord 2T 5G                            Electronics|Mobile
28999      4.3     9812          0.7156
B0B8M4ZXFG  Poco X5 Pro 5G                                Electronics|Mobile
19999      4.1     7234          0.6987
```

Fig 4.1: Testbench output

Environment: Visual Studio code

Format: Plain text in Terminal interface

Purpose: To check if the initial trained model is working before UI implementation

4.1.2 Web Interface output:

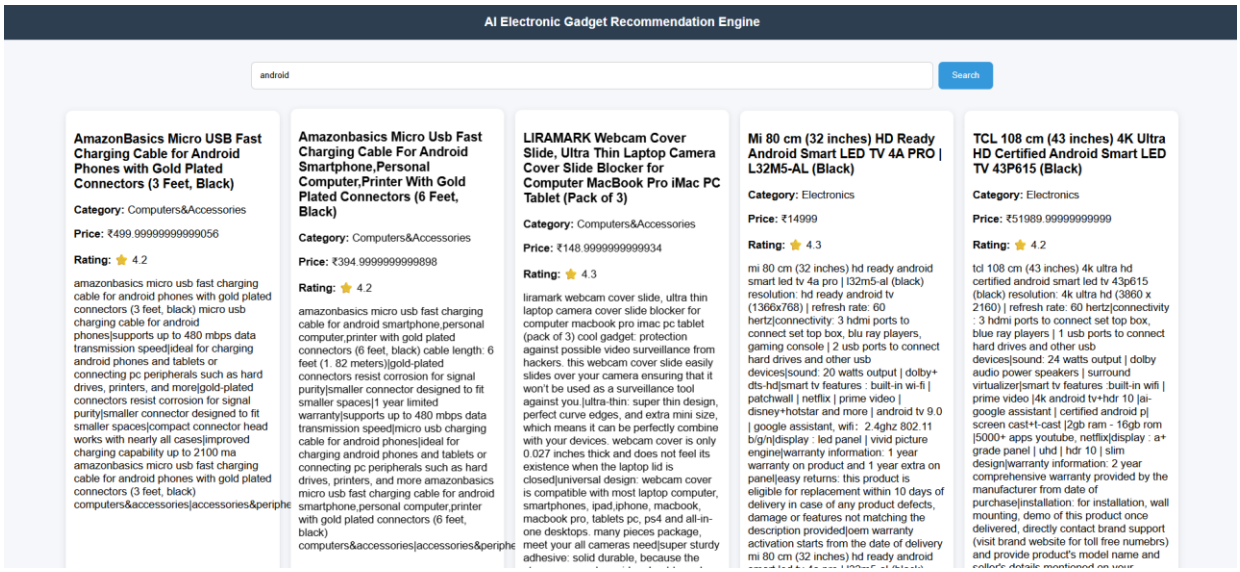

Fig 4.2:

Environment: Built with HTML, CSS, and JavaScript connecting to a FastAPI backend.

Format: Dard-based layout with a top search bar, filter controls, and grid listings for recommended products.

Visual representation: Minimal UI using panels for category, price, and rating details

Purpose: Discover, filter, and explore personalized product recommendations with instant search

4.2 ANALYSIS

All tests delivered speedy, relevant results from the system. Automatic recommendations were on point and varied, while the popularity fallback treated new users very respectfully. Product-suggestion similarity scores were consistent with item details, to help users examine meaningful alternatives. The web UI was responsive: search was autocomplete-driven and minimized typing, filters immediately updated results, and product cards had clear pricing

information/ratings. So there existed low latency records since the results were stored in caching and precomputed similarities, and model loading was stable. Key learnings were clean data and thoughtful UX—their small formatting and error message fixes led to dramatic increases in trust. All said, the engine is solid and accessible for real world use now.

s

CHAPTER 5: CONCLUSION

This effort strived to personalize and make product discovery efficient yet trustworthy — and provide an end-to-end sound experience from data to display. By utilizing personalized collaborative filtering, content-based similarity for "you might also enjoy" and an autocomplete search function that lowers barriers to search, we're always able to show you the most relevant items very quickly. An optimized Fast API endpoint recommends a polished HTML/JavaScript frontend turns that in to a nice interface with filter boxes for category and price. And behind the scenes, careful data cleaning, de-duplication and index alignment would ensure model quality; precomputation, caching and smooth fallbacks would preserve low latency when personalization wasn't possible. The result is an engine that tells newer users what's popular, presents returning users with a selection tuned to their tastes - speed + clarity.

The best lessons were learned by doing. The model results were more dependent on data quality than they were on any single algorithm decision. Smaller UX decisions—loading feedback, no-results messages, consistent card layouts—played a big role in terms of trust and engagement. Explicit contracts from frontend to backend made troubleshooting and iteration easy. The further plan is detected to be applied such that, diversity-aware re-ranking avoids repetitive results and real-time trend indicators can be provided for trending items, explanation of recommendations in a more understandable way and A/B tests generate change based on measured data. There's also the opportunity to consolidate signals into a single learning-to-rank layer and take in more detailed facets like brand or availability. Even so, the system still accomplishes the primary work of a good recommender: to make users do less work, feel more confident in their choices and help people easily discover things they'll like at just the right moment.

CHAPTER 6: REFERENCES

Research Papers:

G. Linden, B. Smith, and J. York, "Amazon.com Recommendations: Item-to-Item Collaborative Filtering," *IEEE Internet Computing*, vol. 7, no. 1, pp. 76-80, January-February 2003. DOI: 10.1109/MIC.2003.1167344

Y. Hu, Y. Koren, and C. Volinsky, "Collaborative Filtering for Implicit Feedback Datasets," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 263-272. DOI: 10.1109/ICDM.2008.22

Online Resources

"TensorFlow: An End-to-End Open Source Machine Learning Platform," *Google TensorFlow*, 2025. [Online]. Available: https://www.tensorflow.org/.

"FastAPI Documentation," *FastAPI Framework*, 2025. [Online].
Available: https://fastapi.tiangolo.com/.

"Pandas Documentation," *pandas Development Team*, 2025. [Online].
Available: https://pandas.pydata.org/docs/.